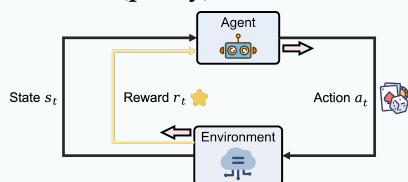


1 Lecture 1: Basic Concepts

1.1 Basic Concepts

Introduction

强化学习研究的是：一个智能体 (**agent**) 如何在不确定环境 (**environment**) 中通过交互 (**action**) 和奖励反馈 (**reward**) 学习一套能最大化长期期望回报的策略 (**policy**)。其是监督学习和非监督学习外第三种基本 ML 方法。



Example: Grid World

一个直观且容易理解的强化学习环境是网格世界 (**Grid World**)。在网格世界中，可以想象有一个机器人 (**agent**) 每次只能移动一格，它的目的是在初始位置 (图1.1中绿色) 以“最好”的方式到达终点 (图1.1中黄色)，中途会遇到一些不能进入障碍物 (图1.1中红色)。其每次移动有五种选择 (如图1.2所示)：上下左右和原地不动。

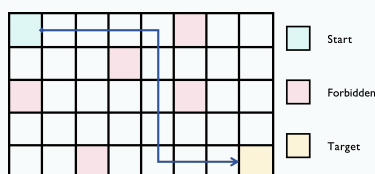


Figure 1.1: Grid world example

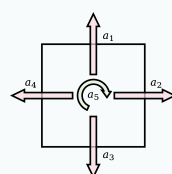


Figure 1.2: Action example

虽然上述例子足够简单，但问题依然可以很复杂，例如 ① 目标是什么？找到一条从起点到达终点的最佳路径还是对任意位置都需要找到最佳路径？② “最好的方式”如何定义？最短的路径、不要撞墙、还是不要撞到障碍物？③ 障碍物是绝对不能进入还是可以但是进入了会有惩罚？惩罚多少？如何施加？④ 当 **agent** 不知晓周围环境时如何规划路径？

State, Action and Policy

State. 状态 (**state**) 是 **agent** 当前情况的反映，其需要包含预测未来状态所需要的全部相关信息，例如位置、当前时间、剩余生命值等。LLM 场景中最自然的 **token-level state** 是 **prompt** + 当前已生成 **tokens**。State 一般记为 s_t 。

State Space. 状态空间 (**state space**) 是有可能状态的集合，一般记为 $\mathcal{S} = \{s_t\}_{t=1}^n$ 。状态空间可以有限离散，例如棋盘上的位置；也可以无限和连续，例如机器人的状态可能包含连续的位置、速度和关节角度。LLM 场景下状态空间近似包含所有可能的 **token** 组合： $\mathcal{S} = \bigcup_{t=0}^L \mathcal{V}^t$ ，其中 $\mathcal{V}$ 为 **vocabulary**， L 为最大上下文长度。(离散巨大)

Action. 动作 (**action**) 是 **agent** 在一定状态下可以执行的行为，其依赖于状态，例如前进方向、速度。LLM 场景下 **token-level action** 是下一个 **token**；**agentic LLM** 中 **action** 也可能是调用某工具。Action 一般记为 a_t 。

Action Space of State. 一定状态下所有可执行动作的集合，与状态 (**state**) 相关，一般记为 $\mathcal{A}(s_t) = \{a_i\}_{i=1}^k$ 。

State Transition. 某一状态采取行动后转变成另一状态，例如可以表示为 $s_1 \xrightarrow{a_2} s_2$ 。其定义了 **agent** 与 **environment** 交互的前后 **state** 的变化，可以用表格 (**tabular**) 的形式展现，例如下面是一个确定性的 **state transition**：

State	a_1 (UPWARD)	a_2 (RIGHTWARD)	a_3 (DOWNWARD)	a_4 (LEFTWARD)	a_5 (STILL)
s_1	s_1	s_2	s_4	s_1	s_1
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	s_6	s_9	s_9	s_8	s_9

State Transition Probability. 由于环境是不确定的，因此采取某 **action** 后 **state** 的变化可能并不是确定性而是使用概率分布进行描述。例如 $P(s_2 | s_1, a_2) = 0.9$ 表示在 **state** s_1 时采取 **action** a_2 变成 **state** s_2 的概率为 90%。

Policy. 指导 **agent** 在某状态时应采取什么行动，通常记为 π 。Policy 的本质是一个从状态到动作概率分布的映射： $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ ，其中 $\Delta(\mathcal{A})$ 表示动作空间上的概率分布，满足 $\pi(a | s) \geq 0$ ， $\sum_{a \in \mathcal{A}(s)} \pi(a | s) = 1$ 。例如在 s_1 时可以

采用 3 种 **action**： $\pi(a_1 | s_1) = 0$ ， $\pi(a_2 | s_1) = 1$ ， $\pi(a_3 | s_1) = 0$ 。这是一个确定性 **policy**，满足 $\pi(a | s) = \begin{cases} 1, & a = \mu(s), \\ 0, & a \neq \mu(s). \end{cases}$

Policy 也可以使用表格 (tabular) 形式展现, 例如下面包括了 deterministic policy 和 stochastic policy:

State	a_1 (UPWARD)	a_2 (RIGHTWARD)	a_3 (DOWNWARD)	a_4 (LEFTWARD)	a_5 (STILL)
s_1	0.2	0.1	0	0.5	0.2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	1	0	0	0	0

Note 1.1. LLM 本身就是一个由 θ 参数化的随机 policy: $\pi_\theta(a_t | s_t) = \pi_\theta(y_{t+1} | x, y_{1:t})$, 它由 logits 经 softmax 得到: $\pi_\theta(a | s) = \frac{\exp(z_\theta(s, a))}{\sum_{a'} \exp(z_\theta(s, a'))}$. 一整段回答的概率是 $\pi_\theta(y_{1:T} | x) = \prod_{t=0}^{T-1} \pi_\theta(y_{t+1} | x, y_{1:t})$.

Reward

Reward. 奖励 (reward) 是环境在某一步给出的标量反馈, 一般记为 $r_t \in \mathbb{R}$. ① Reward 可以依赖于当前状态 s 、动作 a 、下一状态 s' 或它们的组合^a. ② Reward 的相对大小反映了对模型当前 action / policy 的鼓励与否, 一般可以简单认为 $r > 0$ 为鼓励, 反之不鼓励. ③ Reward 是引导 agent 变得越来越好的重要凭据, 其设计需要一定的专家知识, 例如 LLM 的 RLHF 中由人类专家对 LLM 生成答案进行打分. ④ Reward 也可以使用表格形式展现,

State	a_1 (UPWARD)	a_2 (RIGHTWARD)	a_3 (DOWNWARD)	a_4 (LEFTWARD)	a_5 (STILL)
s_1	r_{boundary}	0	0	r_{boundary}	0
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$r_{\text{forbidden}}$	r_{boundary}	r_{boundary}	0	r_{target}

Rewards in LLM

常见的 LLM reward 包括: ① Reward model 打分 (RLHF). 由于 RL 训练数据量巨大, 因此一般通过人类评价一部分回答, 用这些数据训练 reward model, 再由 reward model 来近似人类对回答质量的判断.

② 可验证奖励 RLVR: $R(x, y) = \begin{cases} 1, & \text{答案通过测试,} \\ 0, & \text{否则.} \end{cases}$ 例如数学答案、代码单元测试、形式化证明验证.

③ Agentic LM 工具使用奖励. 例如综合任务是否完成、调用成本、等待时间和错误操作进行打分.

^a也有教程认为 reward 不依赖下一状态 s' , 例如 *Mathematical Foundations of Reinforcement Learning* from Shiyu Zhao.

Trajectory, Episode and Return

Trajectory. 一条轨迹 (trajectory) 是 policy 与 environment 共同作用产生的随机样本, 标准轨迹写为 $\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots)$, 其是一条 state - action - reward chain, 例如: $s_1 \xrightarrow[r=0]{a_2} s_2 \xrightarrow[r=0]{a_3} s_5 \xrightarrow[r=1]{a_3} s_8 \xrightarrow[r=1]{a_2} s_9$.

Note 1.2. ① τ 中大写字母表示随机变量, 因为每个 state, action 和 reward 都具有随机性. 一条确定性的轨迹 τ 的概率为: $P_\pi(\tau) = \rho_0(s_0) \prod_t \pi(a_t | s_t) p(s_{t+1}, r_{t+1} | s_t, a_t)$. ② LLM 中给定 prompt x , 一条 token-level trajectory 写为:

$$s_0 = (x), a_0 = y_1, s_1 = (x, y_1), a_1 = y_2, \dots, a_{T-1} = \text{EOS}.$$

采样出的一条 trajectory 并不足以反映 policy 的好坏, 因为同一个 LLM 重新采样可能得到完全不同的结果. 因此实际训练中需要对多个 prompt (X)、多个 sampled response (Y) 估计平均表现: $J(\pi) = \mathbb{E}_{X \sim \rho_0, Y \sim \pi(\cdot | X)}[G_0]$.

Episode/Trial. 当任务会终止时, 该有限长度的从初始状态开始到 terminal state 结束的完整轨迹被称为 episode/trial: $S_0, A_0, R_1, \dots, S_T, A_T \in \mathcal{S}_{\text{end}}$. 存在 terminal states 的任务被称为 episodic task, 否则为 continuing task.

Return. 回报 (return) 是从时间 t 开始, 未来 reward 的累计量, 因此又称为 total rewards 或 cumulative rewards, 一般记为 G_t . (定义 G_t 为未来而非之前 reward 累积是因为之前轨迹已确定, RL 更关注未来决策.)

Discounted Return. 折扣回报 (discounted return) 定义为 $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$, $\gamma \in [0, 1)$. ① 其目的是避免无限轨迹的 return 无界, 因为若 reward 有界, $|R_t| \leq R_{\max}$ 则 $|G_t| \leq \frac{R_{\max}}{1-\gamma}$. ② LLM 回答通常是 episode, 故一般使用 $\gamma = 1$. 使用 $\gamma < 1$ 反而隐式偏好更短的回答, 并且若最终 reward 为负, 反而会鼓励 agent 拖延坏结果(更接近 0).

1.2 Markov Decision Process

Markov Decision Process (MDP)

马尔可夫决策过程 (Markov decision process, MDP) 是描述序列决策问题的数学模型, 强化学习实际就是在 agent 与环境的不断交互中优化决策过程, 因此可以将其纳入 MDP 框架中. 有限状态、有限动作、带折扣因子的 MDP 通常写成: $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, \mathcal{R}, \rho_0, \gamma)$, 其中 $\gamma \in [0, 1]$ 为折扣因子, ρ_0 为初始状态分布. MDP 框架主要包括:

1. 集合部分: ① state space $\mathcal{S}$ ② action space $\mathcal{A}(s)$, 与状态相关 ③ reward space $\mathcal{R}(s, a)$, 与状态、动作相关.
2. 模型部分: ① State transition probability: 在 state s 采取 action a 后 state 变为 s' 的概率记为 $p(s'|s, a)$, 其需满足 $\sum_{s' \in \mathcal{S}} p(s'|s, a) = 1, \forall (s, a)$ ② Reward probability: 在 state s 采取 action a 后获得 reward r 的概率记为 $p(r|s, a)$, 其需要满足 $\sum_{r \in \mathcal{R}(s, a)} p(r|s, a) = 1, \forall (s, a)$.

MDP 的一个重要性质是马尔可夫性 (Markov property), 是随机过程的一种无记忆属性 (memoryless property): 给定当前状态之后未来的变化不再依赖更早的历史. 这里指下一步的 state, reward 与前面的 state, action 无关:

$$p(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(s_{t+1} | s_t, a_t), \quad p(r_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = p(r_{t+1} | s_t, a_t) \quad (1.1)$$

Note 1.3. ① (s, a) , $p(s'|s, a)$ 和 $p(r|s, a)$ 都是 model 的一部分, 其可以是时变/非稳定的 (time-variant / nonstationary) 也可以是非时变/稳定的 (time-invariant/stationary). 时变指环境可以随时间变化. ② 若 MDP 中的 decision/policy 确定那么其等同马尔可夫过程 (Markov process). 不过注意虽然此时具有概率性的 model 已确定, 但状态序列仍然可能是随机的. 此时 MDP 被诱导为一个由策略决定的 Markov reward process (MRP).

1.3 Questions and Answers

Q & A

Q1: 可以将所有的 reward 设置为负吗?

A1: 可以. 事实上对一个 action 鼓励/不鼓励是由 reward 的相对值而非绝对值决定.

Q2: Reward 是下一 state s' 的函数吗?

A2: 实际上 reward 可以仅依赖当前状态 ($r(s)$), 例如只要当前在危险区域就获得 -1; 也可以依赖当前状态和动作 ($r(s, a)$), 例如工具调用需要付出成本; 也可以依赖完整转移 ($r(s, a, s')$), 例如只有实际进入目标状态才获得奖励. 一种会让人误解的情况是仅关注当前 (s, a) 下的 expected additive return 时, 期望 reward 可以写为:

$$\bar{r}(s, a) = \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{s'} p(s' | s, a) \sum_r r p(r | s, a, s') = \sum_{s'} p(s' | s, a) \bar{r}(s, a, s'). \quad (1.2)$$

此时为了计算期望价值使用了一个经过边缘化的 reward 表示但并不意味原始环境中的 reward 与下一状态无关.